Q:- What is data independence and what are its different types?

→ **Data Independence:**

The separation of data and application program is called data independence. It is an important feature of DBMS, and it is the most important advantage of three level architecture.

There are two types of data independence.

1- Physical Data Independence:

It is a type of independence that enable the user to change the internal level without changing the conceptual level. The changes that may be performed at physical level are as follow:

- Changing file organizations or storage structures
- Using different storage devices
- Modifying indexes
- Changing the access method.

2- Logical data Independence:

It is a type of independence that enables the user to change the conceptual level without changing the external level. The changes that may be performed at this level are as follow:

- Addition or removal of entities or relationships
- Adding a file to the database
- Adding new field in file
- Changing the type of a field etc.

Q2: What is database integrity? How it can be enforced?

- Database Integrity:

Database integrity refers to accuracy consistency and reliability of data stored in database. It ensures that data is valid, complete and follows predefined rules.

→ Ways to enforce database integrity

- Primary key:

Use primary key to identify each row uniquely.

- **Foreign key:**
  Foreign key estables relationship. A relation may contain many foreign keys.
- **Constraints:**
  Integrity contraints are used such as NOT NULL, UNIQUE, CHECK, etc.
- **Normalization:**
  Organize data in order to minimize redundancy.
- **Triggers:**
  Using automated actions before/after the insert, update or delete.

Q: What are limitations of hierarchichal model?

Limitations of hierarchical model include:

- Hierarchichal models can lead to data redundancy, as each parent-child relationship requires repeated data.
- They only support parent-child relationship, making it difficult to represent complex relationships between data entities.
- Querying hierarchical data can be complex and inefficient.
- They have rigid structures, making it challenging to adapt changing business requirements.
- As the database grows, hierarchical models can become unwidely, leading to performance issues and difficulty in maintaining data.

∴ Long 3:

a) List all clerks with SAL between 4000 and 6000.

```
SELECT EMPNO, ENAME, JOB, SAL
FROM EMP
WHERE JOB = 'CLERK'
  AND SAL BETWEEN 4000 AND 6000;
```

b). How many salesman are working in organization.

```
SELECT COUNT (*) FROM EMP
WHERE JOB = 'SALESMAN';
```

c) Display (EMPNO, ENAME, JOB, SAL, DEPTNO, DNAME).

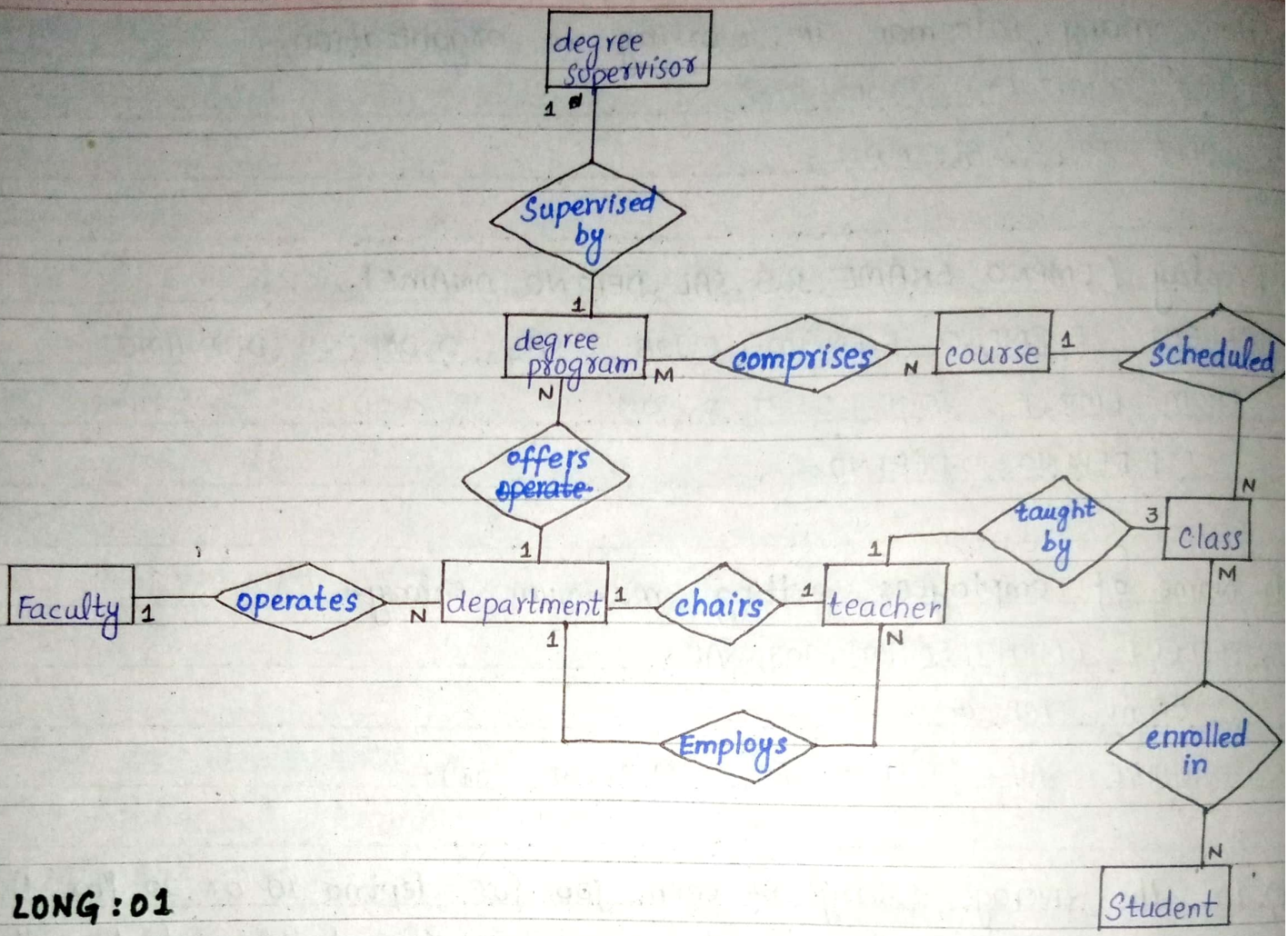
```
SELECT E.EMPNO, E.ENAME, E.JOB, E.SAL, D.DEPTNO, D.DNAME
FROM EMP E JOIN DEPT D ON
    E.DEPTNO = D.DEPTNO;
```

d). Name of employees getting maximum salary.

```
SELECT EMPNO, ENAME, JOB, SAL
  FROM EMP
 WHERE SAL = (SELECT MAX (SAL) FROM EMP);
```

e). List the average salary of each job for deptno 10 or 20, for the jobs with average salary greater than 1500. Sort the output with respect to avg SAL.

```
SELECT JOB, AVG(SAL) AS AvgSalary
FROM EMP
WHERE DEPTNO IN (10,20)
GROUP BY JOB
HAVING AVG(SAL) >1500
ORDER BY AvgSalary;
```



LONG : 01

ERD

→ **LONG Q2 : Normalization:**

Solution: Functional dependencies:

1- ORD_ID 'n ORD_DATE , CUST_ID

2- CUST_ID 'n CUST_NAME, CUST_ADDRESS

3- ITEM_ID 'n ITEM_NAME , ITEM_PRICE

4- ORD_ID , ITEM_ID 'n ITEM_QTY

1NF :

The given table is already in 1NF as it contains only atomic value.

2NF :

We have to eliminate partial dependencies.

ORD_ID , ITEM_ID is composite key.

- **Partial dependencies:**

→ CUST_NAME , CUST_ADDRESS are dependent only on CUST_ID.

→ ITEM_NAME , ITEM_PRICE are dependent only on ITEM_ID.

Split the table :

→ Orders Table

ORD_ID	ORD_DATE	CUST_ID
605	14/03/2021	C-101
605	15/03/2021	C-102

→ Customers table

CUST_ID	CUST_NAME	CUST_ADDRESS
C_101	Naveed	Lahore
C_102	Amjad	Quetta

→ Order_items

ORD_ID	ITEM-ID	ITEM_QTY
605	1	50
605	2	100
605	3	40
606	2	50
606	4	40

→ Items table

ITEM_ID	ITEM-NAME	ITEM-PRICE
1	Scanner	15000
2	Mouse	500
3	Printer	18000
4	Monitor	12000

→ **3NF:** There are no transitive dependencyies as all non-prime attributes are directly dependent on composite key. Thus, tables are already in 3NF.